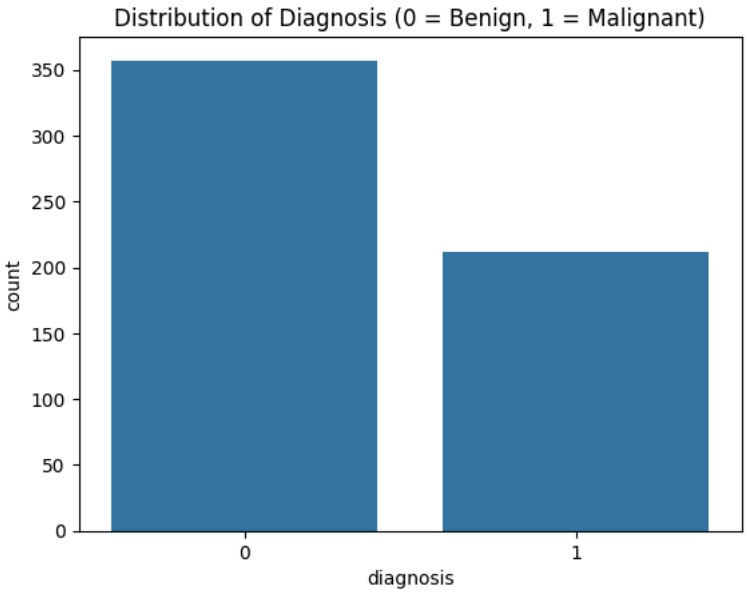
Breast Cancer Detection using ML

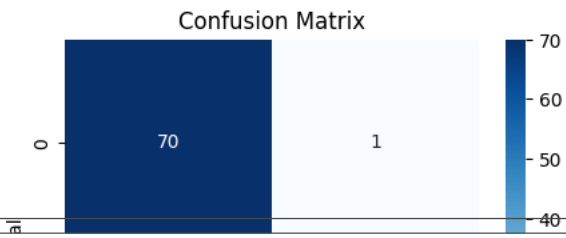
👩‍💼 Developed by: **Parnita**
🎓 AI/ML Intern at **Internpe**
📌 Model: Random Forest Classifier
📊 Dataset: Breast Cancer Wisconsin (Diagnostic) Data
✅ Achieved Accuracy: 96.49%

```
# Step 1: Import Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Step 2: Load the Dataset
df = pd.read_csv('/content/data.csv')
'''print("✅ Dataset Loaded Successfully")'''
df.head()

# Step 3: Data Cleaning
# Remove ID column and unnamed column
df.drop(['id', 'Unnamed: 32'], axis=1, inplace=True)

# Convert diagnosis to binary: M=1, B=0
df['diagnosis'] = df['diagnosis'].map({'M':1, 'B':0})
print("✅ Diagnosis Column Encoded")

# Step 4: Explore the Dataset
print("Dataset Shape:", df.shape)
print("Number of Malignant cases:", df[df['diagnosis']==1].shape[0])
print("Number of Benign cases:", df[df['diagnosis']==0].shape[0])
sns.countplot(x='diagnosis', data=df)
plt.title("Distribution of Diagnosis (0 = Benign, 1 = Malignant)")
plt.show()

# Step 5: Feature Selection & Splitting
X = df.drop('diagnosis', axis=1)
y = df['diagnosis']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
print("✅ Data Split into Training and Testing Sets")

# Step 6: Feature Scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
print("✅ Features Scaled")

# Step 7: Model Training using Random Forest
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
print("✅ Model Trained Successfully")

# Step 8: Model Evaluation
y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
print("Model Accuracy: {:.2f}%".format(accuracy * 100))
print("\nClassification Report:\n", classification_report(y_test, y_pred))

# Confusion Matrix
plt.figure(figsize=(5,4))
sns.heatmap(confusion_matrix(y_test, y_pred), annot=True, cmap='Blues', fmt='d')
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

```
✅ Diagnosis Column Encoded
Dataset Shape: (569, 31)
Number of Malignant cases: 212
Number of Benign cases: 357
```

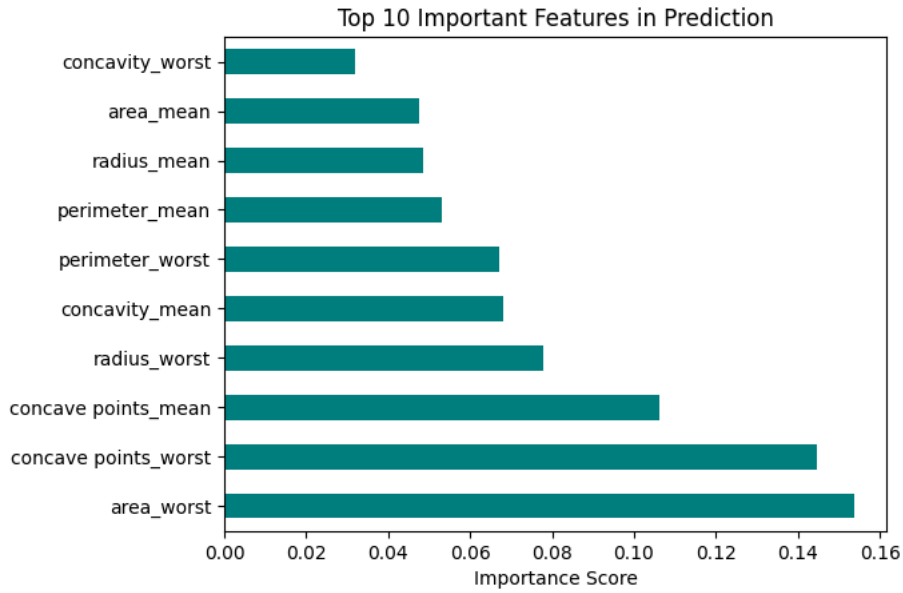



```
✅ Data Split into Training and Testing Sets
✅ Features Scaled
✅ Model Trained Successfully
Model Accuracy: 96.49%

Classification Report:
              precision    recall  f1-score   support

           0       0.96      0.99      0.97        71
           1       0.98      0.93      0.95        43

    accuracy                           0.96       114
   macro avg       0.97      0.96      0.96       114
weighted avg       0.97      0.96      0.96       114
```




```
# Feature Importance
importances = model.feature_importances_
features = X.columns
feat_importance = pd.Series(importances, index=features)
feat_importance.nlargest(10).plot(kind='barh', color='teal')
plt.title("Top 10 Important Features in Prediction")
plt.xlabel("Importance Score")
plt.show()
```

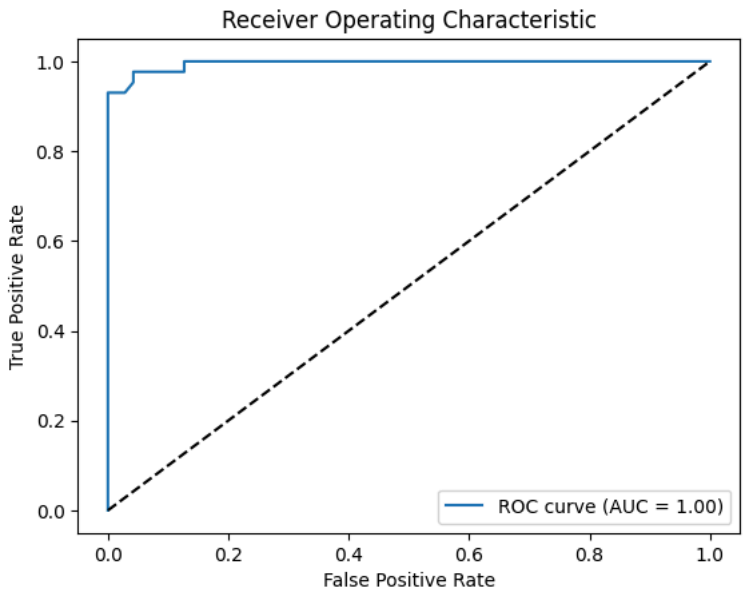
Predicted




```
# Predicting on a single sample
sample = X_test[0].reshape(1, -1)
sample_pred = model.predict(sample)
print("Predicted Diagnosis:", "Malignant" if sample_pred[0] == 1 else "Benign")
```

```
Predicted Diagnosis: Benign
```

```
from sklearn.metrics import roc_curve, auc

y_probs = model.predict_proba(X_test)[:, 1]
fpr, tpr, thresholds = roc_curve(y_test, y_probs)
roc_auc = auc(fpr, tpr)

plt.figure()
plt.plot(fpr, tpr, label='ROC curve (AUC = %0.2f)' % roc_auc)
plt.plot([0, 1], [0, 1], 'k--')
plt.title('Receiver Operating Characteristic')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend(loc="lower right")
plt.show()
```




```
# Test the model on one sample from test set
sample = X_test[0].reshape(1, -1)  # Take first test sample and reshape
sample_pred = model.predict(sample)

print("Predicted Diagnosis for Sample 1:")
```

```
print("🔴 Malignant (Cancerous)" if sample_pred[0] == 1 else "🟢 Benign (Non-Cancerous)")
```

```
Predicted Diagnosis for Sample 1:
🟢 Benign (Non-Cancerous)
```

```
import joblib
joblib.dump(model, 'breast_cancer_rf_model.pkl')
print("✅ Model Saved as 'breast_cancer_rf_model.pkl'")
```

```
✅ Model Saved as 'breast_cancer_rf_model.pkl'
```